HacdsDeFi RPC Documentation

2026-10-21 For developers

0. Services at a glance

hacds involves three HTTP services. They are independent and can be deployed separately.

Service	Default address	Protocol	Purpose
Full node	`43.251.101.101:8082`	HTTP/JSON	Chain data, sandbox read-only calls, transaction verify/broadcast
Oracle	`43.251.101.101:8123`	HTTP/JSON	Signs diamond scoring certificates
Indexer	`43.251.101.101:8124`	HTTP/JSON	Borrower ledger (who can be liquidated)

The wallet does not scan the chain locally. All chain data comes through these three services.

```
full node :8082 ──GET──► indexer :8124 ──GET /watchlist──► client
      ▲                                                      │
      └──────────────── oracle :8123 (signs certs) ◄─────────┘
```

Response convention: every endpoint returns JSON with a top-level `ret` field. `ret: 0` means success; non-zero means failure (usually with an `err` field).

Note: failures also return **HTTP 200**. You must check `ret` in the body — the status code alone will not tell you.

---

1. Full node API

1.1 Block height

```
GET /query/block/height/latest
```

```
{"ret":0,"height":792580}
```

1.2 Average fee

For fee estimation.

```
GET /query/fee/average
```

```
{"ret":0,"purity":"6024000000","purity_unit":232}
```

`purity` = `fee / billing_size (bytes)`. Node admission requires `purity ≥ min_purity` (currently `6024000000`).

1.3 Balance (HAC / diamonds / assets)

```
GET /query/balance?address={address}&assets=true&asset_meta=true&diamonds=true
```

All three switches are optional:

Parameter	Effect
`assets=true`	Returns the `assets` array (on-chain assets)
`asset_meta=true`	Adds `name` / `ticket` / `supply` / `decimal` to each asset
`diamonds=true`	Returns `diamonds` (a concatenated name string, 6 characters per diamond)

Measured response (address `15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`):

```
{
  "ret": 0,
  "list": [{
    "hacash": "217001914996:237",
    "diamond": 7,
    "satoshi": 0,
    "diamonds": "ZUKKXZTTBAAUAYEWKKIXAYYUYTXNNBEUETIVYBYHYX",
    "assets": [{
      "serial": "2000",
      "amount": "4000000000",
```

```
      "decimal": 8,
      "name": "hacds",
      "ticket": "HACDS",
      "supply": "5963298700000000"
    }]
  }]
}
```

Field	Meaning
`hacash`	HAC balance, formatted `value:unit` (`237` means 1:237 precision)
`diamond`	Number of diamonds
`diamonds`	Concatenated diamond names, **6 characters each**
`satoshi`	SAT balance
`assets[].serial`	Asset serial. **hacds =** `2000`
`assets[].amount`	Balance. A string, to avoid JS precision loss
`assets[].decimal`	Decimal places. hacds is `8`
`assets[].supply`	Total supply

⚠ `amount` **is in atoms, not a count.** `4000000000` is 40 hacds. See section 4.

1.4 Diamond details

```
GET /query/diamond?name={6-character diamond name}
```

```
{
  "ret": 0,
  "name": "KWTVSH",
  "belong": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "inscriptions": ["hacds"],
  "number": 85662,
  "miner": "1GSL5G6LyuKgg325DxPtfPouaTeTHaZbGh",
  "born": {"height": 522125, "hash": "00000000..."},
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "visual_gene": "aab018d628cb912cc290"
}
```

`belong` is the current owner. If it equals the contract address, the diamond has been deposited.

`visual_gene` is the input for the artwork and the score.

1.5 Sandbox read-only call (the main way to read contract state)

```
GET /query/contract/sandboxcall?contract={contract}&function={function}&params={params}&caller={caller}
```

This executes read-only: it does not write state, does not create a transaction, and costs no fee. But for calls that read contract data, **pass the contract address itself as** `caller` — otherwise some functions fail because the caller has no balance (see the note below).

Measured response:

```
{"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}
```

Field	Meaning
`ret`	0 = the call succeeded
`ret_val`	The function's return value (u128, string or number)
`use_gas` / `gas_use`	Consumption for this call, for estimation

⚠ **Note 1: omitting** `caller` **fails.** Measured without it:

```
{"ret":1,"err":"[revert] address \"1111111111111111111114oLvT2\" balance 0:0 is insufficient, at least 559555:242"}
```

The default caller has zero balance and cannot cover the gas pre-charge. **Pass** `caller={contract address}` **and it works.**

⚠ **Note 2: per-IP concurrency is capped at 2.** The node limits concurrent sandbox calls, and requests over the cap are **rejected outright** (no queueing):

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

Clients must **serialize and retry on failure**. Do not fire a dozen at once with `Promise.all`.

Read-only contract functions

Parameters are formatted `value:type`, comma-separated for multiple.

Function	Parameters	Returns	Notes
`amm_reserves`	—	u128	HAC in the pool (**zhu**)
`amm_reserve_ds`	—	u128	hacds in the pool (**atoms**)
`pool_price`	—	u128	Pool price = `amm:hac × 1e8 / amm:ds`
`lp_total`	—	u128	Total LP shares
`lp_share_of`	`{addr}:address`	u128	LP shares of an address
`lp_pending_hac`	`{addr}:address`	u128	Pending LP reward (HAC, zhu)
`lp_pending_ds`	`{addr}:address`	u128	Pending LP reward (hacds, atoms)
`total_staked`	`{kind}:u128`	u128	Total staked. kind `0` =HAC, `1` =hacds
`treasury_pending`	`{kind}:u128`	u128	Treasury pending. kind `0` =HAC, `1` =hacds
`balance_hac`	`{addr}:address`	u128	In-contract HAC points (zhu)
`balance_ds`	`{addr}:address`	u128	Pending hacds quota, `due` (atoms)
`loan_status`	`{addr}:address`	u128	Borrower status, see below
`loan_debt_of`	`{addr}:address`	u128	Debt (zhu)
`loan_coll_of`	`{addr}:address`	u128	Collateral (atoms)
`quote_score`	`{name}:bytes,{vg}:bytes,{num}:u128`	u128	Score for a name + gene + number

`loan_status` **return semantics** (important — do not read it as a health percentage):

Value	Meaning
`0`	No loan, no collateral
`999`	Has collateral but no debt (safe)
anything else	`coll_hac × 100 / debt`, i.e. **collateral ratio ×100**

Liquidation predicate: liquidatable when `debt > 0 && 0 < status < 200`.

Measured samples:

```
GET /query/contract/sandboxcall?contract=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
    &function=amm_reserves&caller=VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
→ {"ret":0,"use_gas":377,"gas_use":{"compute":132,"resource":209,"storage":36},"ret_val":120200677111}

GET ...&function=loan_status&params=15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q%3Aaddress&caller={contract}
→ {"ret":0,"use_gas":830,"gas_use":{"compute":363,"resource":323,"storage":144},"ret_val":0}
```

> `:` must be URL-encoded as `%3A`.

1.6 Block intro

The indexer uses this to walk blocks looking for borrowers.

```
GET /query/block/intro?height={height}&tx_hash_list=true
```

```
{
  "ret": 0,
  "hash": "0000000000279d55a83105ce01311cd50b06fb5efcb1175e3891a9b46fcf27b1",
  "version": 1,
  "height": 792575,
  "timestamp": 1792537829,
  "nonce": 17,
  "difficulty": "...",
  "miner": "...",
  "reward": "...",
  "tx_hash_list": ["b6488be2b07be8745ac5827e00928ebc840a803d6a132cd9eb573812221e55ce"]
}
```

`tx_hash_list` is only returned when `tx_hash_list=true` is passed.

1.7 Transaction lookup

```
GET /query/transaction?hash={tx hash}&action=true
```

Without `action=true`, only the summary comes back:


```
{
  "ret": 0,
  "hash": "b6488be2...",
  "hash_with_fee": "cf947b86...",
  "type": 2,
  "timestamp": 1792537576,
  "fee": "245:247",
  "fee_got": "245:246",
  "main_address": "1NsStAq1xt7cuqvEAobcS8T6gqcyS2gnMB",
  "action": 1,
  "block": {"height": 792575, "timestamp": 1792537829},
  "confirm": 5
}
```


With `action=true`, a structured action array is added:


```
"actions": [{
  "kind": 4,
  "d": {
    "diamond": "IKEMYN",
    "number": 138592,
    "prev_hash": "0x0000...",
    "nonce": "0x00000001a90d5c45",
    "address": "1EaqdAi9FLCQwLYPLWVtoRM1fZwGezXLTi",
    "custom_message": "0xb0114b5f..."
  }
}]
```


Contract calls are `kind = 44`, and their bytecode contains the function selector:

```
09 14 01 <sig4: 4 bytes> ee
```

`sig4 = sha3_256(function name)[0:4]`. This is how the indexer identifies which contract function a transaction called.

Note: the transaction must be confirmed (`confirm ≥ 1`) for this endpoint to find it; unconfirmed ones are not visible.

1.8 Verify a transaction (with human-readable description)

```
POST /util/transaction/check?description=true
Content-Type: application/octet-stream
Body: the transaction's raw bytes (not a hex string)
```

Signed or unsigned both work. With `description=true` it returns plain-language descriptions.

Measured response:


```
{
  "ret": 0,
  "hash": "37637b89...",
  "hash_with_fee": "14f0c5e3...",
  "type": 3,
  "timestamp": 1792469415,
  "fee": "1:248",
  "fee_got": "1:248",
  "main_address": "15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q",
  "action": 1,
  "gas_max": 99,
  "description": "Main account 15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q pay 1:248 HAC tx fee",
  "actions": [{
    "action": {
      "kind": 16,
      "metadata": {
        "serial": 2000,
        "supply": 5963298700000000,
        "decimal": 8,
        "issuer": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
        "ticket": "0x4841434453",
        "name": "0x6861636473"
      },
      "protocol_cost": "8:248"
    },
    "description": "Register Asset #2000 \"HACDS\" (name \"hacds\", supply 5963298700000000 atoms, 8 decimals, issuer VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1);
pay 8 HAC protocol fee"
```

```
  }]
}
```

Failures also return HTTP 200:

```
{"ret":1,"err":"transaction body invalid"}
```

1.9 Broadcast a transaction

```
POST /submit/transaction
Content-Type: application/octet-stream
Body: the signed transaction's raw bytes
```

Success:

```
{"ret": 0, "hash": "tx hash"}
```

Failure:

```
{"ret": 1, "err": "transaction parse failed"}
```

⚠ `ret:0` **only means it reached the mempool, not that it is on chain.** Hacash targets **300-second** blocks, so it takes effect only at the next block.

To confirm, poll `/query/transaction?hash=...` and watch the `confirm` field, or watch the balance via `/query/balance`.

1.10 Contract storage fee parameters

Used when deploying a contract or estimating the cost of new keys.

```
GET /query/contract/storage_fee
```

```
{
  "ret": 0,
  "enabled": true,
  "rule_version": 1,
  "head_height": 792581,
  "target_capacity_blocks": 1000,
  "capacity": 1000000,
  "rate": 1000,
  "remaining": 1000000,
  "p_min": 10,
  "p_max": 10000,
  "periods": 10,
  "k_max": 16384
}
```

`periods` is the current discount period count. If it is not at full price, regenerate the deployment action with it.

---

2. Oracle API

2.1 Health check

```
GET /health
```

```
{"ok": true, "oracle": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu"}
```

`oracle` is the oracle account address. **This address is fixed in the contract** — certificates signed by any other address are not accepted.

2.2 Request a scoring certificate

Required to deposit or redeem diamonds.

```
GET /cert?name={6-character diamond name}
```

Measured response:

```
{
  "name": "KWTVSH",
  "number": 85662,
  "visual_gene": "aab018d628cb912cc290",
```

```
  "life_gene": "05e7b8f313761bc8d55ddfe6945b3ba1aedbb4065288bceb5901d24c3cb2a9aa",
  "score": 20,
  "hacds_amount": 2000000000,
  "details": {
    "shapes": "Common diamond",
    "styles_or_color": "Double mix",
    "nameliteral": "Common diamond",
    "numberliteral": "Common diamond",
    "score": 20
  },
  "cert_message_hex": "4b5754565348aab018d628cb912cc2900000000000014e9e0000001401",
  "cert_sha3_256": "0fca486f6d7c934006efb7ab926a12928bfda49d09ced3440c27ac2c39a88921",
  "sign_hex":
"023d7424448df611baf27f5188d1e48450082e94bb4d207b1612e22d299bd372d1aff139d57bd79446670c3ef77bab7471a33ada1df5f72adc90e13d316d7bcffc4ce07221ae7730e4b84b51f6320
  "oracle_address": "1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu",
  "version": 1
}
```


Field	Meaning
`score`	Diamond score. **1 point = 1 hacds**
`hacds_amount`	hacds granted, in **atoms** (`2000000000` = 20)
`cert_message_hex`	Certificate message body (29 bytes), goes into the transaction's Message/Blob
`sign_hex`	The oracle's signature over that message (97 bytes)
`oracle_address`	The signing account

Certificate length = 126 bytes (29-byte message + 97-byte signature). N diamonds in one transaction means `N × 126` bytes of certificates.

Errors:

```
{"error": "query name must be 6 chars"}   // HTTP 400
{"error": "..."}                          // HTTP 502, usually the node is unreachable
```

⚠ **The diamond name must be exactly 6 characters.** Case-insensitive (the server upper-cases it).

---

3. Indexer API

The indexer only consumes the full node read-only. It **never submits transactions, needs no private key, and needs no balance.**

It solves one problem: **the contract cannot enumerate borrowers**. Loan records are stored under keys built from the address (`loan:debt:<address>`); the contract has no list, writes no log, and the node cannot enumerate contract KV by prefix. So the only way is to walk blocks and dig the addresses that ever called a lending function out of history.

> **Deployment status**: currently `43.251.101.101:8124` returns `502` — the service is not running.
> The response shapes below are taken from the source (`hacds_indexer.py`); they are what you get once it runs.

3.1 Health check

```
GET /health
```

```
{"ok": true, "service": "hacds-indexer", "head": 792580, "watched": 12}
```

3.2 Ledger snapshot (the only endpoint a client needs)

```
GET /watchlist
```

```
{
  "ret": 0,
  "head": 792580,
  "node": "http://43.251.101.101:8082",
  "contract": "VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1",
  "watched": 12,
  "withDebt": 3,
  "liquidatable": 1,
  "snapshot": [{
    "address": "1Abc...",
    "health": 150,
    "healthPct": 1.5,
    "debt": 12345,
    "coll": 67890,
    "liquidatable": true,
    "noLoan": false,
    "first": 792000,
```

```
    "last": 792575,
    "calls": 4,
    "fns": {"loan_borrow": 2, "loan_add_collateral": 2},
    "manual": false,
    "note": null
  }]
}
```

Field	Meaning
`head`	Height scanned to
`watched`	Number of addresses in the ledger
`withDebt`	How many have debt
`liquidatable`	How many are liquidatable
`snapshot[].health`	Raw `loan_status` (`0` / `999` / ratio ×100)
`snapshot[].healthPct`	Converted to a multiple (`1.5` = 150%). `null` when `0` or `999`
`snapshot[].liquidatable`	Already judged server-side by `debt > 0 && 0 < health < 200`
`snapshot[].first/last`	First / most recent block height it appeared in a lending call
`snapshot[].fns`	Call counts per function
`snapshot[].manual`	Whether it was added by hand

Clients do not need to compute liquidatability themselves — use `liquidatable` directly.

3.3 Snapshot only

```
GET /snapshot
```

```
{"ret": 0, "list": [ /* same snapshot array as above */ ]}
```

3.4 Sync now

```
GET /sync
```

Triggers one scan and returns the result.

3.5 Maintaining the ledger by hand

For early days when there is no history to scan, or to add addresses you want watched.

```
GET /watch/add?address={address}&note={note}
GET /watch/remove?address={address}
```

`/watch/add` returns the updated address list:

```
{"ret": 0, "list": ["1Abc...", "1Def..."]}
```

`/watch/remove`:

```
{"ret": 0, "removed": true}
```

Missing `address` returns HTTP 400:

```
{"ret": 1, "err": "address required"}
```

⚠ **The ledger must be single-source.** If you run more than one indexer instance, `/watch/add` and `/watch/remove` must go to the same one, or the ledgers diverge.

3.6 Deployment

```
python3 hacds_indexer.py --port 8124 --node http://<node>:8082
```

- Zero third-party dependencies, Python 3 standard library
- `--workers` for concurrent scanning (default 4; measured 2.6 → 6.6 blocks/s)
- Incremental: records `lastScanned`, then only scans new blocks
- Ledger written atomically (`tmp` + `os.replace`)
- systemd unit at `hacds_exchange/indexer/hacds_indexer.service`

4. Exchange listing integration (dedicated chapter)

This section is for exchanges, wallets and market-data services integrating hacds.

4.0 Remember one thing first: amount is in atoms

hacds is a `decimal = 8` asset. The `amount` on chain and in the interfaces is the **smallest unit, atoms**, not a count.

```
1 hacds = 100000000 atoms (1e8)
total supply = 5963298700000000 atoms = 59,632,987 hacds
```

Getting this wrong means being off by a factor of one hundred million.

4.1 Asset identity (for listing registration)

Item	Value
serial	**2000**
ticket	`HACDS`
name	`hacds`
decimal	**8**
total supply	`5963298700000000` atoms
issuer	`VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1`
issuer address	`15fv6cNEapTNqYhigMPr38pC1XRKn3Wh5q`

⚠ There is another asset on chain whose ticket is also `HACDS` (serial `1028`, name "HACD Fragments", unrelated to us).

The protocol does not check ticket or name uniqueness — it only checks whether the serial is taken.

Match on `serial = 2000`. Do not match on ticket.

4.2 Deposit detection

Do not match on serial alone — check serial=2000 *and* the issuer.

Poll for blocks or subscribe, then parse the asset actions in each transaction:

```
GET /query/block/intro?height={height}&tx_hash_list=true
  → tx_hash_list
GET /query/transaction?hash={hash}&action=true
  → actions[], filter for asset transfers
```

Asset transfer action `kind`:

kind	Action	Meaning
17	`TransferAssetTo`	Transfer to the contract / a simple target
19	`TransferAssetFromTo`	Transfer from A to B

Read `serial` / `amount` and the from/to addresses from the action's `d` field. `serial == 2000` **is hacds**, and it is advisable to also check that `issuer` is the contract address.

Amount to credit:

```
hacds count the user sees = amount / 1e8
```

For example, receiving `amount = 4000000000` credits **40 hacds**.

⚠ Use big integers. `5963298700000000` is 66% above JS `Number.MAX_SAFE_INTEGER` (`9007199254740991`). The `amount` in JSON is a string — **do not parse it into a double first**. Use `BigInt` in JS, `big.Int` in Go, `BigInteger` in Java, `BigInteger` in Kotlin (`Long` is not enough).

4.3 Constructing withdrawals

hacds is a standard fungible asset, so a withdrawal is an ordinary asset transfer (`kind = 19`, `TransferAssetFromTo`).

Parameters:

Parameter	Value
`serial`	`2000`
`amount`	atoms to withdraw (= count × 1e8)
`from`	exchange hot wallet address

Parameter	Value
`to`	user address

Note: `from` must be the transaction's initiating account, or otherwise authorised; the exchange has to manage its hot wallet key and sign locally.

Two ways to build it:

1. Build the Type-3 transaction yourself with **Hacash's official tooling**
2. **Refer to this repo's Rust engine** `hacds_txbuild` (the `transfer asset-from-to` submode); the source already handles Type-3 wire format, fee iteration and signing

> We do not provide a remote signing endpoint. **The node's `/util/transaction/sign` requires sending your private key over HTTP** — never use it under any circumstances.

4.4 Broadcast and confirmation

```
POST /submit/transaction       # binary body, not hex
```

```
{"ret": 0, "hash": "..."}
```

Confirmation flow:

1. `ret:0` → it is in the mempool, **not yet confirmed**
2. Poll `GET /query/transaction?hash={hash}`
3. Watch the `confirm` field; `≥ 1` means it is on chain
4. Or simply check the balance change via `GET /query/balance`

Block target is 300 seconds. Design your deposit/withdrawal risk confirmations around that rhythm. Measured on chain today: roughly one block every 5 minutes.

4.5 Fees and gas

- **Fee**: charged by transaction byte size. Get `purity` from `GET /query/fee/average`; `fee = purity × billing_size`.

Raising the fee slightly increases the byte size (the fee encoding gets longer), so **iterate**: build → read the actual byte size → rebuild if short. `fee.js` in this repo is that iteration; it usually converges in 1–3 rounds.

- **gas**: the `gas_max` byte, default `99` which is the protocol cap `111,911`. Exchange transfer transactions do not touch the contract, so gas consumption is very low.

4.6 Three hard limits you must know

① VM gas has three independent buckets, unrelated to `gas_max`

Bucket	Cap
compute	18,009
resource	6,100
storage	111,911

When exceeded, **raising `gas_max` does nothing**. The error looks like:

```
OutOfGas(31): resource gas limit exceeded: used 6101 > limit 6100
```

② At most one HACD per contract deposit transaction

Verifying each diamond's certificate requires on-chain score recomputation, about 3,000 resource; two diamonds come to 6,101 > 6,100.

If the exchange offers "diamonds for hacds", it **must split into one diamond per transaction**. Merging with an atomic transaction does not help either (they share one gas budget).

③ Sandbox calls are capped at 2 concurrent per IP

Do not parallelise contract-state reads; requests over the cap are rejected outright (no queueing):

```
{"ret":1,"err":"sandbox_per_ip_concurrency_exceeded"}
```

4.7 Data needed for market making / quotes

What you need	Endpoint
HAC in the pool	`sandboxcall` → `amm_reserves` (zhu; ÷1e8 = HAC)
hacds in the pool	`sandboxcall` → `amm_reserve_ds` (atoms)

What you need	Endpoint
Spot price	`sandboxcall` → `pool_price` = `amm:hac × 1e8 / amm:ds`, i.e. **HAC per hacds**
Total supply	`supply` of serial 2000 from `/query/balance?address={contract}&assets=true`
Liquidatable positions	indexer `GET /watchlist`

Quote conversion:

```
pool price in HAC per hacds = pool_price / 1e8
e.g. pool_price = 301355749 → 3.01355749 HAC/hacds
```

4.8 Reconciliation

An on-chain identity you can check at any time:

```
user holdings + contract holdings = total supply
```

All three numbers are retrievable:

```
# total supply
GET /query/balance?address={contract}&assets=true     → supply
# contract holdings
GET /query/balance?address={contract}&assets=true     → amount
# user holdings
GET /query/balance?address={user}&assets=true         → amount
```

Measured sample (2026-10-21):

Item	atoms
total supply	`5963298700000000`
contract holdings	`5963294700000000`
user holdings	`4000000000`
contract + user	`5963298700000000` ✓

4.9 Listing integration checklist

- [ ] Match the asset by **serial 2000**, and also verify the issuer
- [ ] Keep `amount` in big integers throughout; never through a double
- [ ] Divide by `1e8` when displaying a count to users
- [ ] Detect deposits by parsing asset actions for `kind 17/19`
- [ ] Build withdrawals as `TransferAssetFromTo(2000, amount, hotWallet, user)`
- [ ] Broadcast with a binary body to `POST /submit/transaction`
- [ ] Confirm on `confirm ≥ 1`, allowing for the 5-minute block delay
- [ ] Estimate fees iteratively (`purity × size`)
- [ ] Serialize sandbox reads (per-IP concurrency ≤ 2)
- [ ] If diamonds are involved, **one per transaction**
- [ ] Reconcile with the identity "user + contract = total supply"

---

5. Error handling

5.1 Contract return codes

When a sandbox call returns a non-zero `ret_val`, it is the contract function's internal `return N`. Common mappings:

Function	Code	Meaning
`swap_hac_to_ds`	11	Insufficient HAC points
`add_liquidity`	2	One side is 0
`add_liquidity`	23	Pool non-empty but shares zeroed (rescale guard)
`add_liquidity`	24	Computed shares are 0 (ratio too small)
`add_liquidity`	26	First deposit: one side exceeds the `1e18` cap
`remove_liquidity`	25	Insufficient LP shares

Function	Code	Meaning
`loan_borrow`	40	Exceeds borrow limit (`(debt+hac)×2 > coll_hac`)
`loan_repay`	41	Debt is 0
`loan_liquidate`	42	The address has no debt
`loan_liquidate`	43	Not past the liquidation line (cannot liquidate)
`Set*` (admin)	1	Not the admin
`Set*` (admin)	2	Parameter out of range
`PermitAsset`	11	Insufficient pending hacds
`PermitHACD`	12	Insufficient hacds for redemption and auto top-up failed

The full table is in `wallet-app/contract-errors.js` .

5.2 Node / sandbox errors

Matched by string. The common categories:

Raw message	Meaning	Handling
`invalid address argument` / `InvalidChecksum`	Address format or checksum wrong	Re-fetch the full address
`balance ... is insufficient`	Caller cannot cover the gas pre-charge	Pass `caller={contract address}` on sandbox calls
`sandbox_per_ip_concurrency_exceeded`	Concurrency over the cap	Serialize and retry
`resource gas limit exceeded`	resource bucket exceeded	Split the transaction; **raising gas_max does not help**
`compute gas limit exceeded`	compute bucket exceeded	Split the transaction
`storage gas limit exceeded`	storage bucket exceeded	Fewer new keys, or a shorter rental period
`OutOfGas`	Out of gas	Raise gas_max
`CallNotExist`	No such function in the contract	Check the function name
`NotFindContract`	Contract not found	Check the contract address
`state changed`	Chain state changed meanwhile	Retry
`transaction parse failed`	Transaction body failed to parse	hex / bytes are incomplete
`ECONNREFUSED` / `timeout`	Service unreachable	Check whether the service is running

Key point: most of these come back as **HTTP 200 +** `{"ret":1,"err":"..."}` . Checking only the HTTP status will miss them.

---

6. Quick reference

Endpoints

```
# full node :8082
GET  /query/block/height/latest
GET  /query/fee/average
GET  /query/balance?address=&assets=true&asset_meta=true&diamonds=true
GET  /query/diamond?name=
GET  /query/contract/sandboxcall?contract=&function=&params=&caller=
GET  /query/block/intro?height=&tx_hash_list=true
GET  /query/transaction?hash=&action=true
GET  /query/contract/storage_fee
POST /util/transaction/check?description=true     # binary body
POST /submit/transaction                          # binary body

# oracle :8123
GET  /health
GET  /cert?name=

# indexer :8124
GET  /health
GET  /watchlist        # the only one a client needs
GET  /snapshot
GET  /sync
GET  /watch/add?address=&note=
GET  /watch/remove?address=
```

Constants

```
hacds serial         2000
hacds decimal        8
hacds total supply   5963298700000000 atoms
1 hacds              100000000 atoms
contract             VKNwXFFMCEn9XfMrhyHhd5rHqBnEyM3m1
treasury             1FbwjELGMmaomNQq3ZZ6hdgQ17GRGwzAzK
oracle account       1H3sFZ53HzNVUVN97mjpC9JT8s2PqKMyRu
SCORE_SCALE          1e8 (1 point = 1 hacds)
COLL_RATIO           2 (50% collateral ratio, liquidation line 200)
AMM fee              0.5% (LP 50% / staking 30% / treasury 20%)
lending rate / liq bonus  5% APR / 2%
certificate length   126 bytes (message 29 + signature 97)
VM bucket caps       compute 18,009 / resource 6,100 / storage 111,911
block target         300 seconds
sandbox concurrency  per IP 2
```

Unit conversion

```
atoms → hacds count : amount / 1e8
hacds count → atoms : count × 1e8
zhu   → HAC         : zhu / 1e8
pool price (HAC/hacds) : pool_price / 1e8
```